ELM264 ÖDEV 2

AD SOYAD: FOUAD ALADHAMI
Öğrenci No: 210102002808

İçindekiler:

I. Problem 1. Birleşik Devreler ve Aritmetik 3

A. Signed sayılarla işlem yapabilen bir ALU devresinde aşağıdaki Tabloda verilen 16-bit operasyonlara göre sonuçların N-Z-V-C bayrak değerlerini hesaplayınız. 3

B. ALU devresi sonuçlarını gösteren bu NZVC bayrakları baz alarak unsigned HS, LS, HI, ve LO çıkışlarını gösteren devrelerin boolean denklemlerini bulun. Devrenin veri girişlerini 16-bit olarak alın. 6

C. Problem 1B'de elde edilen Boole cebri bayrak fonksiyonlarını DTD ile tasarlayın ve simüle edin. Test tezgahında A ve B girişlerine bayrak çıkışlarını test edecek şekilde giriş değerleri uygulayın. 7

D. 67.5123 sayısını IEEE 754 floating point standardına uygun şekilde hem single hem double precision olarak ikilik düzende gösteriniz. 10

E. -152.23813 IEEE 754 floating point standardına uygun şekilde hem single hem double precision olarak ikilik düzende gösteriniz. 12

..... 13

F. C. ve D. deki problemlerde verilen iki sayıyı IEEE 754 gösteriminde toplayınız ve sonucu ikilik düzende gösteriniz. 14

II. Makale Özetleme 16

I. Problem 1. Birleşik Devreler ve Aritmetik

A. Signed sayılarla işlem yapabilen bir ALU devresinde aşağıdaki Tabloda verilen 16-bit operasyonlara göre sonuçların N-Z-V-C bayrak değerlerini hesaplayınız.

bayrak değerlerini hesaplayınız.

	operasyon
a	0x1F32 - 0x FF02
b	0x0010 + 0x1F01
c	0xFF01 + 0x7000
d	0xFE02 - 0x00DF
e	0x3D20 + 0xF5D0

a) 0x1F32 = 0001 1111 0011 0010

0xFF02 = 1111 1111 0000 0010

0x1F32 - 0xFF02 değerini bulmak için

-0xFF02 (0xFF02 two's complementi dir) ona bakalım

1 yerine 0, 0 yerine bir koyalım

0000 0000 1111 1101

ve bir ekleyelim + 1

-0xFF02 = 0000 0000 1111 1110

toplayalım (0x1F32) + (-0xFF02)

```
  0001 1111 0011 0010
+ 0000 0000 1111 1110
-------------------
  0010 0000 0011 0000
```

= 0x2030

- N bayrağı sayının negatif olup olmadığını temsil eder N=0 çünkü işaret biti 0.
- Z (zero) bayrağı sayı 0 olursa bir olur o yüzden Z=0
- V (Adder overflowed) Aynı işarete sahip 2 sayı toplanırken sonuç işaret bitini değiştirecek kadar büyük veya küçük bir değer getirse 1 olur (taşmayı düşünün mesela). Burda V=0.
- C (Carry out) toplama sırasında taşıma olunca 1 olur burda taşıma olmadığı için C=0

b. 0x0010 + 0x1F01

0x0010 = 0000 0000 0001 0000

0x1F01 = + 0001 1111 0000 0001

0001 1111 0001 0001

0 olmadığı için Z=0 | işaretler aynı V=0

Negatif değil N=0 | Taşıma yok C=0

c. 0xF801 + 0x7000

111 1

0xF801 = 1111 1000 0000 0001

0x7000 = + 0111 0000 0000 0000

1 0110 1000 0000 0001

0 değil Z=0 -2047 + 28672 = 26625

Negatif değil N=0

V=0

Ama taşıma var ondandolayı

C=1 'dir

d) 0xFF02 - 0x000F

0xFF02 = 1111 1111 0000 0010

0x000F = 0000 0000 1101 1111

-0x000F değerini bulmaki için 0x000F'i

ters çevirip bir eklegelim

1111 1111 0010 0000

+ 1

1111 1111 0010 0001

1 1 1 1 1 1 1 1

0xFF02 = 1111 1111 0000 0010

+ -0x000F = 1111 1111 0010 0001

1 1111 1110 0010 0011

0 olmadığı için Z=0

V=0

Negatiftir N=1

Taşıma var C=1 'dir

e) 0x3020 + 0xF500

0x3020 = 0011 1101 0010 0000

0xF500 = 1111 0101 1101 0000

---

1 0011 0010 1111 0000

Z=0 çünkü 0 değil

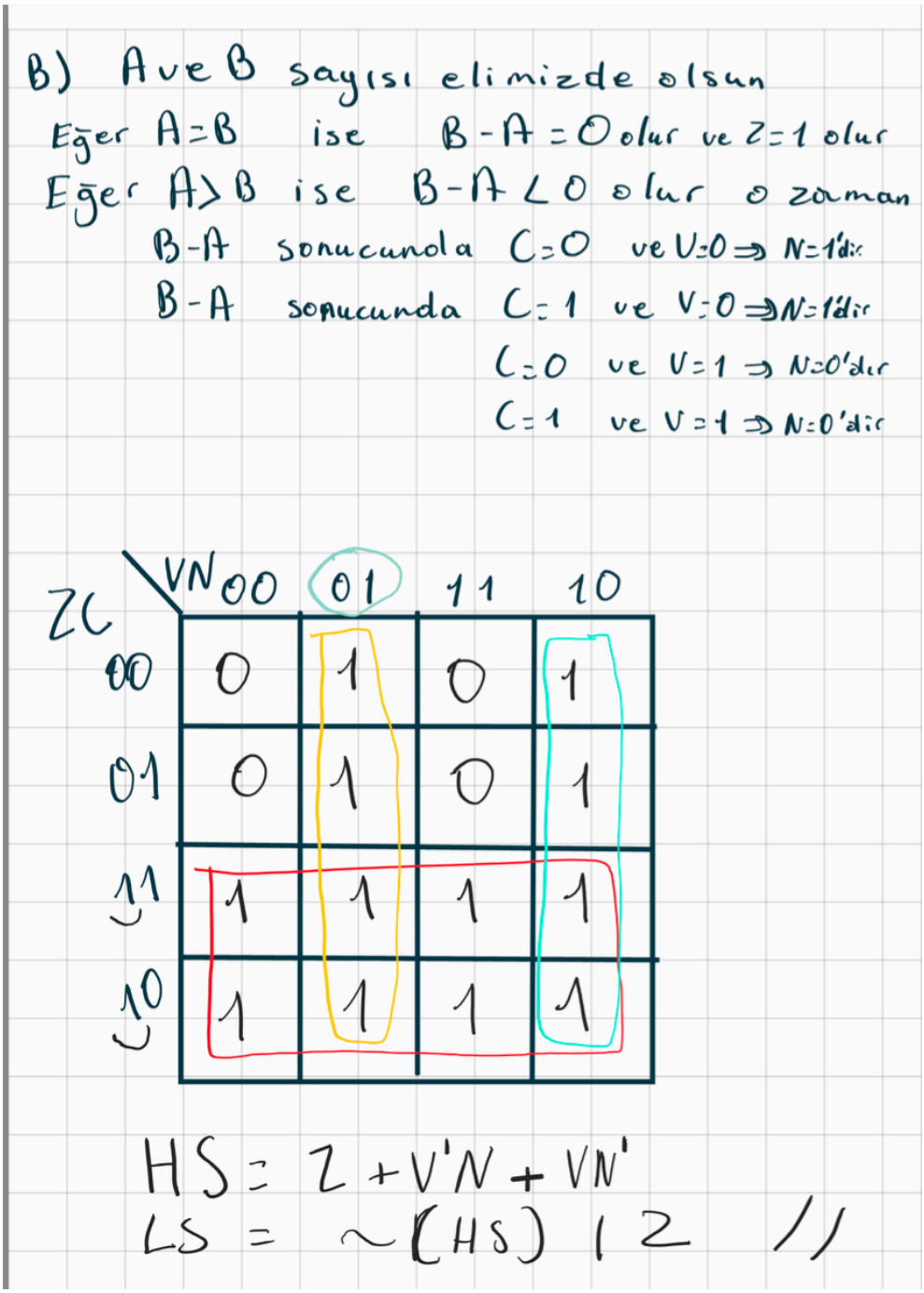
Negatif değil N=0

Aynı işarete sahip iki sayı farklı işaret çıkarırsa V 1 olur
Burda V=0

Taşıma var C=1

B. ALU devresi sonuçlarını gösteren bu NZVC bayrakları baz alarak unsigned HS, LS, HI, ve LO çıkışlarını gösteren devrelerin boolean denklemlerini bulun. Devrenin veri girişlerini 16-bit olarak alın.

A ve B sayıları 16 bitlik signed integer olmak üzere, A – B işlemindeki bayrakları kullanarak oluşturulan HS, LS, HI ve LO boole denklemeleri aşağıdaki gibidir.
HS = NOT(N XOR V) = ~(N^V)
LS = (N XOR V) OR Z = (N^V) + Z
HI = NOT(LS) = NOT(Z) AND NOT(N XOR V) = ~Z & ~(N XOR V)
LO = NOT(HS) = N XOR V = (N^V)
Elle yaptığım işlem ise aşağıdaki gibidir:



C. Problem 1B'de elde edilen Boole cebri bayrak fonksiyonlarını DTD ile tasarlayın ve simüle edin. Test tezgahında A ve B girişlerine bayrak çıkışlarını test edecek şekilde giriş değerleri uygulayın.

```
//alu.sv
module ALU (
  input signed  [15:0] operandA,
  input signed [15:0] operandB,
  input signed [3:0] operation,
  output logic N, Z, V, C,
  output  logic signed [15:0] result,
  output logic HS, LS , HI ,LO
);

  always @(*) begin
    case (operation)
      4'b0000: begin  // Operation: Subtract
        result = operandA - operandB;
        N = result[15];
        Z = (result == 0);
        V = (operandA[15] == operandB[15] && operandA[15] != result[15]);
        C = (operandA < operandB);
        HS = ~(N^V);
        LS = N^V | Z;
        HI = ~LS;
        LO = ~HS;

      end

      4'b0001: begin  // Operation: Add
        result = operandA + operandB;
        N = result[15];
        Z = (result == 0);
        V = (operandA[15] == operandB[15] && operandA[15] != result[15]);
        C = (result < operandA);

      end

      default: begin
        result = 16'hxxxx;
        N = 1'b0;
        Z = 1'b0;
        V = 1'b0;
        C = 1'b0;
      end
    endcase
  end
endmodule
```

```
/*iverilog kodu çalıştırma komutu
iverilog -g2012 -o tb_alu tb_alu.sv alu.sv
vvp tb_alu

*/
module tb_alu();
  logic signed [15:0] operandA, operandB;
  logic [3:0] operation;
  logic N, Z, V, C;
  logic signed [15:0] result;
  logic unsigned HS, LS , HI ,LO;
  logic signed [15:0] A ,B;
  ALU uut (
    .operandA(operandA),
    .operandB(operandB),
    .operation(operation),
    .N(N),
    .Z(Z),
    .V(V),
    .C(C),
    .result(result),
    .HS(HS),
    .LS(LS),
    .HI(HI),
    .LO(LO)

  );

  initial begin
    $dumpfile("tb_alu.vcd");//modelsim ile çalışırken bu  ve sonraki satır
silenebilir
    $dumpvars(0, tb_alu);
    // Test case A
    operandA = 16'h1F32;
    operandB = 16'hFF02;
    A = operandA;
    B = operandB;
    operation = 4'b0000; // Subtract
    #10; // Wait for ALU computation
    $display("Test case A:");
    $display("Result: %h", result);
    $display("N: %b, Z: %b, V: %b, C: %b", N, Z, V, C);
    $display("HS: %b, LS: %b, HI: %b, LO: %b", HS, LS, HI, LO);
    $display("operandA: %h, operandB: %h", operandA, operandB);
    $display("A: %h, B: %h", A, B);
    $display("expected HS: %b, LS: %b, HI: %b, LO: %b", (A>B ) , (A<B  ),
(A>B & A!=B) , (A<B& A!=B) );
```

```
    // Test case B
    operandA = 16'hF801;
    operandB = 16'h7000;
    A = operandA;
    B = operandB;
    operation = 4'b0000; // Subtract
    #10; // Wait for ALU computation
    $display("Test case B:");
    $display("Result: %b", result);
    $display("N: %b, Z: %b, V: %b, C: %b", N, Z, V, C);
    $display("HS: %b, LS: %b, HI: %b, LO: %b", HS, LS, HI, LO);
    $display("expected HS: %b, LS: %b, HI: %b, LO: %b",  A>B | A==B , A<B | B==A
, A>B & A!=B , A<B & A!=B);
    // Test case C
    operandA = 16'hFE02;
    operandB = 16'h00DF;
    A = operandA;
    B = operandB;
    operation = 4'b0000; // Subtract
    #10; // Wait for ALU computation
    $display("Test case C:");
    $display("Result: %h", result);
    $display("N: %b, Z: %b, V: %b, C: %b", N, Z, V, C);
    $display("HS: %b, LS: %b, HI: %b, LO: %b", HS, LS, HI, LO);
    $display("expected HS: %b, LS: %b, HI: %b, LO: %b", A>B | A==B , A<B | B==A
, A>B & A!=B , A<B& A!=B);
    $finish;
  end
endmodule
```

D. 67.5123 sayısını IEEE 754 floating point standardına uygun şekilde hem single hem double precision olarak ikilik düzende gösteriniz.

Binary sayıları decimale çevirmek zor olacağından python kullanarak belli çözünürlükte binary sayıları decimale , decimal sayıları binariye, binary'ye çeviriken olacak hatayı hesaplayan , aynı zamanda önceki fonksiyonları kullanarak IEEE754 standadına göre çeviren bir program yazdım.
Program ilk öce decimal sayıyı girdi olarak almaktadır. 'decimal_to_ieee754' fonksiyonun içine decimal sayı ve single yada double presicion olup olmadığını belirtmek için ikinci parametre olarak 'single' yada 'double' kelimleri verilmektedir. Bu fonksiyon ilk önce decimal sayıyı binaryi nereye çevirmektedir istenen çözünürlükte (single için 23 bit , double için 52 bit mantissa (anlamlı kısım çözünürlüğü)).
Aşağıda IEEE754 standardı hakkında gerekli açıklma kağıt üzerinde yapılmıştır.

Kod ise bu şekildedir:

```
# -*- coding: utf-8 -*-
def decimal_to_binary(decimal, resolution):
    binary = ""
    integer_part = int(decimal)
    fractional_part = decimal - integer_part

    # Convert integer part to binary
    binary += bin(integer_part)[2:]
    #fraction resolution assigned
    fractionres = resolution - len(binary)
    # Convert fractional part to binary
    binary += "."

    for _ in range(fractionres):
        fractional_part *= 2
        bit = int(fractional_part)
        binary += str(bit)
        fractional_part -= bit

    return binary

def binary_to_decimal(binary, resolution):
    integer_part, fractional_part = binary.split(".")

    # Convert integer part to decimal
    decimal = int(integer_part, 2)
    #fraction resolution assigned
    fractionres = resolution - len(integer_part)
    # Convert fractional part to decimal
    for i in range(fractionres):
        bit = int(fractional_part[i])
        decimal += bit / 2**(i+1)

    return decimal

def calculate_error(decimal, binary, resolution):
    converted_decimal = binary_to_decimal(binary, resolution)
    error = abs(decimal - converted_decimal)
    return error
def decimal_to_ieee754(decimal , representation = 'single'):
    if representation == 'single':
        resolution = 23 + 1

    elif representation == 'double':
        resolution = 52 + 1
    else: return 'representation error'
    binary = decimal_to_binary(decimal,resolution)
    converted_decimal = binary_to_decimal(binary, resolution)
    error = calculate_error(decimal, binary, resolution)
    exponent = str(len(binary.split('.')[0])-1)
    base = '2'
    mantissa = binary.split('.')[0][0] + '.' + binary.split('.')[0][1:] + binary.split('.')[1]
    return (binary, converted_decimal, error, mantissa+'x'+ base + '^' + exponent)

decimal = 67.5123
binary,converted_decimal,error, ieeenumber = decimal_to_ieee754(decimal)
print('Single presicion için işlemler\n')
print(f"Binary: {binary},\nConverted Decimal: {converted_decimal},\nError: {error},\nIEEE754 single
representation: {ieeenumber}")

binary,converted_decimal,error, ieeenumber = decimal_to_ieee754(decimal,'double')
print('Double presicion için işlemler\n')
print(f"Binary: {binary},\nConverted Decimal: {converted_decimal},\nError: {error},\nIEEE754 double
representation: {ieeenumber}")
```

Kodun çıktısı ise bu şekildedi :
Single presicion için işlemler

Binary: 1000011.10000011001001100,
Converted Decimal: 67.51229858398438,
Error: 1.4160156212028596e-06,
IEEE754 single representation: 1.00001110000011001001100x2^6
Double presicion için işlemler

Binary: 1000011.10000011001001100001011111000001101111011010 01,
Converted Decimal: 67.5123,
Error: 0.0,
IEEE754 double representation:
1.00001110000011001001100001011111000001101111011010 01x2^6

E. -152.23813 IEEE 754 floating point standardına uygun şekilde hem single hem double precision olarak ikilik düzende gösteriniz.

D şıkkında yazılan kod güncellenerek kullanılmıştır. Kod aşağıdaki gibidir:

```
# -*- coding: utf-8 -*-
def decimal_to_binary(decimal, resolution):
    binary = ""
    integer_part = int(decimal)
    fractional_part = decimal - integer_part
    # Convert integer part to binary
    binary += bin(integer_part)[2:]
    #fraction resolution assigned
    fractionres = resolution - len(binary)
    # Convert fractional part to binary
    binary += "."
    for _ in range(fractionres):
        fractional_part *= 2
        bit = int(fractional_part)
        binary += str(bit)
        fractional_part -= bit

    return binary

def binary_to_decimal(binary, resolution):
    integer_part, fractional_part = binary.split(".")

    # Convert integer part to decimal
    decimal = int(integer_part, 2)
    #fraction resolution assigned
    fractionres = resolution - len(integer_part)
    # Convert fractional part to decimal
    for i in range(fractionres):
        bit = int(fractional_part[i])
        decimal += bit / 2**(i+1)

    return decimal

def calculate_error(decimal, binary, resolution):
    converted_decimal = binary_to_decimal(binary, resolution)
    error = abs(decimal - converted_decimal)
    return error
def decimal_to_ieee754(decimal , representation = 'single'):
    if representation == 'single':
        resolution = 23 + 1

    elif representation == 'double':
        resolution = 52 + 1
    else: return 'representation error'
    binary = decimal_to_binary(decimal,resolution)
    converted_decimal = binary_to_decimal(binary, resolution)
    error = calculate_error(decimal, binary, resolution)
    exponent = str(len(binary.split('.')[0])-1)
    base = '2'
    mantissa = binary.split('.')[0][0] + '.' + binary.split('.')[0][1:] +
binary.split('.')[1]
    return (binary, converted_decimal, error, mantissa+'x'+ base + '^' + exponent)

#for negative number
decimal = abs(-152.23813 )
binary,converted_decimal,error, ieeenumber = decimal_to_ieee754(decimal)
print('Single presicion için işlemler\n')
print(f"Binary: -{binary},\nConverted Decimal: -{converted_decimal},\nError:
{error},\nIEEE754 single representation: -{ieeenumber}")

binary,converted_decimal,error, ieeenumber = decimal_to_ieee754(decimal,'double')
print('Double presicion için işlemler\n')
print(f"Binary: -{binary},\nConverted Decimal: -{converted_decimal},\nError:
{error},\nIEEE754 double representation: -{ieeenumber}")
```

Kodun çıktısı ise aşağıdaki gibidir:

Binary: -10011000.0011110011110110,
Converted Decimal: -152.23812866210938,
Error: 1.3378906373873178e-06,
IEEE754 single representation: -1.00110000011110011110110x2^7
Double presicion için işlemler

Binary: -10011000.00111100111101100001011001110010001100100101 0,
Converted Decimal: -152.23813,
Error: 0.0,
IEEE754 double representation: -
1.0011000001111001111011000010110011100100011001001010x2^7

F. C. ve D. deki problemlerde verilen iki sayıyı IEEE 754 gösteriminde toplayınız ve sonucu ikilik düzende gösteriniz.

67.5123 = *1.000 0111 0000 0110 0100 1100x2^6*
-152.23813 = *-1.001 1000 0011 1100 1111 0110x2^7*
Birinci adım üstleri ve kesir (virgülden sonraki kısmı) ayıralım.
6 üstünü göstermesi için 127(biase) + 6 yaparız, 127 +6 = 133 = 10000101
7 üstünü göstermesi için 127(biase) + 7 yaparız, 127 +7 = 134 = 10000110

İşaret biti	Üst	Kesir
0	1000 0101	*1 . 000 0111 0000 0110 0100 1100*
1	1000 0110	*1. 001 1000 0011 1100 1111 0110*

1000 0110
- 1000 0101
0000 0001 (kaydırma mıktarı)
2.kesir bölümünü sağa bir bit kaydıralım

İşaret biti	Üst	Kesir
0	1000 0110	*0. 100 0011 1000 0011 0010 0110*
1	1000 0110	*1. 001 1000 0011 1100 1111 0110*

Büyük olanın işareti eksi olduğundan çıkış de eksidir.
Eksi olan sayının two' complementini alalım
1. 001 1000 0011 1100 1111 0110 'in two's complementi aşağıdaki gibidir (Ters çevir 1 ekle)
0. 110 0111 1100 0011 0000 1010

İşaret biti	Üst	Kesir
0	1000 0110	*0. 100 0011 1000 0011 0010 0110*
0	1000 0110	0. 110 0111 1100 0011 0000 1010

Toplayalım

0. 100 0011 1000 0011 0010 0110
0. 110 0111 1100 0011 0000 1010
=
1. 010 1011 0100 0110 0011 0000
Sağa bir kaydıralım ve üste ekleyelim

İşaret biti	Üst	Kesir
1	1000 0110	1. 010 1011 0100 0110 0011 0000

Burda üst yanlış oldu 6 olması gerekirken 7 olsu,onun için başka bir yöntem denedim ve orda başarılı oldu.

2. deneme
67.5123 = *1.000 0111 0000 0110 0100 1100x2^6*
-152.23813 = *-1.001 1000 0011 1100 1111 0110x2^7*
Birinci adım üstleri ve kesir (virgülden sonraki kısmı) ayıralım.
6 üstünü göstermesi için 127(biase) + 6 yaparız, 127 +6 = 133 = 10000101
7 üstünü göstermesi için 127(biase) + 7 yaparız, 127 +7 = 134 = 10000110

İşaret biti	Üst	Kesir
0	1000 0101	*1 . 000 0111 0000 0110 0100 1100*
1	1000 0110	*1. 001 1000 0011 1100 1111 0110*

1000 0110
- 1000 0101
0000 0001 (kaydırma mıktarı)
2.kesir bölümünü sağa bir bit kaydıralım

İşaret biti	Üst	Kesir
0	1000 0110	*0. 100 0011 1000 0011 0010 0110*
1	1000 0110	*1. 001 1000 0011 1100 1111 0110*

Çıkarma yapalım 0.101 0100 1011 1001 1101 0000

O zaman sonuç

İşaret biti	Üst	Kesir
1	1000 0101	1.010 1001 0111 0011 1010 0000

O'da decimal olarak -84.725830078125'e denk gelmektedir

II. Makale Özetleme

Bu makalede, günümüzde kullanılan transistörlerin sınırlamaları ve yeni çözüm yolları konuları ele alınmaktadıe. Makale, Moore Kanunu'nun evrimsel yolculuğunun son halkası olarak nanosheet cihazlarının öne çıktığı bir gelecek öngörmektedir.

Çünkü, yarıiletken endüstrisindeki ilerlemelerin hızını belirleyen Moore Kanunu'nun, geleneksel transistörlerin daha fazla küçültülmesiyle artık sürdürülemez bir noktaya ulaştığıdır. Bugün kullanılan transistörler, küçülme süreciyle birlikte karşılaşılan çeşitli sorunlarla karşı karşıyadır. En önemliler, ısıl dağılımın kontrolü ve enerji tüketimidir, çünkü bunlar yarıiletken endüstrisinde sınırlamalara yol açmaktadır. Bunun için çözümler üretilmeye çalışılmaktadır. 2011 yılına kadar en iyi transistör Planer FET'idi. Sonrasında ise FinFET'e geçildi. FinFET'te kanal(Channel) daha ince , ve kanal Gate ile çevrili olacak şekilde tasarlanmıştır böylece daha iyi bir kontrol ve daha az istenmeyen akım (leakage current) akmasını sağlar. Ama bu 3nm transistör yapabilmek için uygun değildir, çünkü tasarımsal olarak limitlere sahiptir. Daha iyi bir kontrol ve ısı kullanımı için kanal boyunu büyütmrk gerekir.

Yakın zamanda ortaya çıkan çözüm yolları arasında nanosheet cihazlar ön plana çıkmaktadır. Samsung endüstrinin yaklaşık on yıldır kullanmakta olduğu transistör türünün ömrünü tamamladığına karar verdi. Ve 2020 den beri 3nm yeni transistörler elde etmek için çalışmalara koyuldu.

Nanosheet transistörleri, finFET (Fin Field Effect Transistor) transistörlerinden daha iyi performans sunabilen üç boyutlu bir mimariye sahiptir.Bu transistörün kanal kısmını oluşturmak için, üst üste dizilmiş silicon ve silicon germanyum levhalar konulur, özel bir kimyasal ile silicon germanyum aşındırılır, Gate dielectric malzemeyle kaplanır, sonrasında ise metal gate eklenir. Bu cihazlar, daha iyi kontrol edilebilir ısıl dağılım ve enerji verimliliği sağlayarak, yarıiletken endüstrisinin sınırlamalarını aşmayı hedeflemektedir.

Geleneksel transistörlerin küçülmesiyle birlikte, daha yoğun bir şekilde paketlenen devrelerde ısının kontrolü zorlaşmaktadır. Bu, sıcaklık artışı, güvenilirlik sorunları ve enerji tüketimi gibi ciddi sınırlamalara yol açmaktadır. Nanosheet cihazlarının geliştirilmesiyle birlikte, daha düşük ısıl dağılım ve daha az enerji tüketimi gibi avantajlar elde edilebilir.

Makale aynı zamanda ileriye dönük teknolojik gelişmelere dair öngörüler sunmaktadır. Nanosheet cihazlarının kullanılmasıyla, yarıiletken endüstrisindeki ilerleme sürecinin yeniden canlanması ve Moore Kanunu'nun bir süre daha devam etmesi beklenmektedir. Ayrıca, makale, transistörlerlerde kullanılması için germaniyium, indium arsenide ve gallium antimonide üerinde araştırmalar yapıldığına değinmektedir. Nanosheet kullanımı gelecekteki transistörleri inşa etmenin en iyi yolu olarak gözüktüğünü vurgulamaktadır.